CONFIGURACIÓN Y PREPARACIÓN PARA GOOGLE PLAY STORE

Desarrollo de Aplicaciones Móviles:

NIVEL 4

Nombre: Gemma Serrano Ruiz

Aplicación: MiAppNivel4

Curso: Fundación Estatal para la Formación en el Empleo (Fundae)

ÍNDICE

1. INTRODUCCIÓN Y DESCRIPCIÓN GENERAL 4
1.1. OBJETIVO DEL PROYECTO 4
2. TECNOLOGÍAS Y HERRAMIENTAS UTILIZADAS 4
2.1. ENTORNO DE DESARROLLO 4
2.3. SERVICIO WEB Y PERSISTENCIA 5
2.4. COMPONENTES PRINCIPALES DE ANDROID 7
3. DISEÑO DE LA INTERFAZ DE USUARIO 7
3.1. PANTALLA PRINCIPAL Y ANIMACIONES 7
3.2. ADAPTACIÓN A LA ORIENTACIÓN DE PANTALLA 8
3.3. PERSONALIZACIÓN VISUAL 9
4. ARQUITECTURA Y FUNCIONALIDADES PRINCIPALES 9
4.1. ORGANIZACIÓN DE LAS TAREAS 9
4.2. AÑADIR TAREAS 9
4.3. ELIMINAR UNA TAREA INDIVIDUAL 9
4.4. ELIMINAR TODAS LAS TAREAS 9
4.5. NAVEGACIÓN A LA PANTALLA DE DETALLE 10
4.6. SALIDA DE LA APLICACIÓN 10
5. CONSUMO DE LA API Y PERSISTENCIA DE DATOS 10
5.1. OBTENCIÓN DE TAREAS MEDIANTE DUMMYJSON 10
5.2. IMPLEMENTACIÓN DE LA PERSISTENCIA LOCAL 10
6. PRUEBAS Y VALIDACIÓN DEL PROYECTO 11
6.1. PRUEBAS FUNCIONALES 11
6.2. CONFIGURACIÓN Y VALIDACIÓN DE LA VERSIÓN RELEASE 11
6.3. GENERACIÓN DE LA APK 12
6.4. INSTALACIÓN EN UN DISPOSITIVO FÍSICO 12
7. PREPARACIÓN DE LA APLICACIÓN PARA SU DISTRIBUCIÓN 12
7.1. GENERACIÓN DE UN APK FIRMADO 12
7.2. GENERACIÓN DE UN ANDROID APP BUNDLE (AAB) 14
7.3. CONSIDERACIONES DE SEGURIDAD 15
8. PROCEDIMIENTO DE PUBLICACIÓN EN GOOGLE PLAY CONSOLE 15
8.1. REGISTRO Y ACCESO A GOOGLE PLAY CONSOLE 15
8.2. CREACIÓN DE LA APLICACIÓN 16

8.3. CUMPLIMENTACIÓN DE LOS REQUISITOS DE LA APLICACIÓN 16
8.4. CARGA DEL PAQUETE DE DISTRIBUCIÓN 16
8.5. PRUEBAS PREVIAS A LA PUBLICACIÓN 17
8.6. REVISIÓN Y PUBLICACIÓN 17
9. PREPARACIÓN DE LA FICHA DE GOOGLE PLAY 17
9.1. DATOS GENERALES 17
9.2. DESCRIPCIONES DE LA APLICACIÓN 18
9.3. RECURSOS GRÁFICOS 18
9.4. INFORMACIÓN LEGAL Y PRIVACIDAD 20
10. CONCLUSIÓN 20

CONFIGURACIÓN Y PREPARACIÓN PARA GOOGLE PLAY STORE

Desarrollo de Aplicaciones Móviles: NIVEL 4

MiAppNivel4

Este manual describe cómo se prepararía la aplicación para su distribución en Google Play Store. La aplicación realmente no se va a publicar, los apartados de Play Console son una guía del procedimiento.

1. INTRODUCCIÓN Y DESCRIPCIÓN GENERAL

- **Nombre del Proyecto:** MiAppNivel4
- **Nombre de la aplicación visible para el usuario:** Mis Tareas
- **Tipo de Proyecto:** Ejercicio práctico para curso de formación de Fundae.
- **Lenguaje de programación:** Java
- **Entorno de desarrollo:** Android Studio

1.1. OBJETIVO DEL PROYECTO

Permite consultar tareas de ejemplo, añadir tareas propias, eliminar una tarea mediante pulsación larga, vaciar la lista y abrir una pantalla de detalle al seleccionar un elemento. También conserva los cambios en el almacenamiento local y adapta la interfaz a las orientaciones vertical y horizontal.

El objetivo de este documento es explicar cómo está configurado el proyecto, qué función cumple cada componente y qué pasos se seguirían para preparar un paquete y completar el flujo de distribución en Google Play Console, sin efectuar una publicación real.

2. TECNOLOGÍAS Y HERRAMIENTAS UTILIZADAS

2.1. ENTORNO DE DESARROLLO

- **Android Studio:** entorno utilizado para crear, configurar, desarrollar y ejecutar la aplicación.
- **Java:** lenguaje empleado para implementar la lógica de la aplicación.
- **XML:** utilizado para definir las interfaces, los componentes visuales y los recursos gráficos.

- **Gradle**: herramienta de configuración y compilación del proyecto**.**

2.2. LIBRERÍAS Y DEPENDENCIAS

- **RecyclerView:** componente utilizado para mostrar la lista de tareas de forma eficiente y permitir la interacción con cada elemento.
- **RecyclerView.Adapter y ViewHolder:** empleados para asociar los datos de las tareas con sus elementos visuales.
- **Retrofit:** librería utilizada para realizar peticiones HTTP a un servicio web.
- **Gson:** librería empleada para convertir datos entre objetos Java y formato JSON.
- **SharedPreferences:** mecanismo de almacenamiento local utilizado para conservar la información de las tareas entre ejecuciones.

```
implementation(libs.gson) com.google.code.gson:gson:2.14.0

implementation(libs.retrofit) com.squareup.retrofit2:retrofit:3.0.0
implementation(libs.retrofit.converter.gson) com.squareup.retrofit2:converter-gson:3.0.0
```

Fig 1. Dependencias GSON y Retrofit

2.3. SERVICIO WEB Y PERSISTENCIA

Se utiliza **DummyJSON** como API REST para proporcionar un conjunto inicial de tareas de ejemplo. Mediante **Retrofit** se gestionan las peticiones HTTP -empleando diferentes acciones como GET, POST, etc.- y se procesan las respuestas para extraer los datos que se muestran en la interfaz.

GSON convierte la colección de tareas a JSON y permite reconstruirla al iniciar la aplicación. **SharedPreferences** almacena ese JSON en el dispositivo bajo una clave identificativa. Cuando el usuario añade o elimina tareas, la aplicación debe guardar el estado actualizado.

```
public interface ApiService {

    1 usage
    @GET("todos")
    Call<TodosResponse> getTasks();

    1 usage
    @POST("todos/add")
    Call<Todo> addTask(@Body Todo task);
}
```

Fig. 2. Interface ApiService para Retrofit

```java
    private void loadTasks() {
        SharedPreferences preferences = getSharedPreferences( name: "tasks", MODE_PRIVATE);
        String json = preferences.getString( s: "task_list",  s1: "");

        if (!json.isEmpty()) {
            try {

                Type type = new TypeToken<ArrayList<String>>() {
                }.getType();

                ArrayList<String> savedTasks = new Gson().fromJson(json, type);

                if (savedTasks != null && !savedTasks.isEmpty()) {
                    taskList.clear();
                    taskList.addAll(savedTasks);
                    taskAdapter.notifyDataSetChanged();
                }else{
                    preferences.edit().remove( s: "task_list").apply();
                    loadTasksFromApi();
                }

            }catch (Exception e){
                preferences.edit().remove( s: "task_list").apply();
                loadTasksFromApi();
            }
        } else {
            loadTasksFromApi();

        }
    }
```

Fig. 3 y 4. Código en Java que muestra la lógica de recuperación local de tareas y la sincronización con el servidor externo

```java
private void loadTasksFromApi() {

    apiService.getTasks().enqueue(new Callback<TodosResponse>() {

        @Override
        public void onResponse(Call<TodosResponse> call, Response<TodosResponse> response) {
            // Comprobamos si la respuesta es exitosa
            if (response.isSuccessful()) {
                if (response.body() != null && response.body().getTodos() != null) {
                    android.util.Log.d( tag: "API_DEBUG",  msg: "¡Conexión OK! Tareas recibidas: " + response.body().getTodos().size());

                    taskList.clear();
                    for (Todo todo : response.body().getTodos()) {
                        taskList.add(todo.getTodo());
                    }

                    taskAdapter.notifyDataSetChanged();
                    saveTasks();
                } else {
                    android.util.Log.d( tag: "API_DEBUG",  msg: "Error: La respuesta del servidor vino vacía (body o todos es null)");
                }
            } else {
                android.util.Log.d( tag: "API_DEBUG",  msg: "Error HTTP del servidor: " + response.code());
            }
        }

        @Override
        public void onFailure(Call<TodosResponse> call, Throwable t) {
            Toast.makeText( context: MainActivity.this,
                    text: "No se pudieron cargar las tareas",
                    Toast.LENGTH_SHORT).show();
        }
    });
}
```

2.4. COMPONENTES PRINCIPALES DE ANDROID

Durante el desarrollo se han utilizado, entre otros, los siguientes componentes:

- Activity para gestionar las diferentes pantallas.
- ConstraintLayout para organizar los elementos de la interfaz mediante restricciones.
- RecyclerView para mostrar el listado de tareas.
- EditText para introducir nuevas tareas.
- Button para ejecutar las acciones principales.
- TextView e ImageView para mostrar textos e imágenes.
- Intent para navegar desde la pantalla principal hasta la pantalla de detalle.

3. DISEÑO DE LA INTERFAZ DE USUARIO

3.1. PANTALLA PRINCIPAL Y ANIMACIONES

La pantalla principal, identificada como Mis Tareas, incorpora un icono y un título que cuentan con animaciones de movimiento, proporcionando una presentación dinámica al iniciar la aplicación.

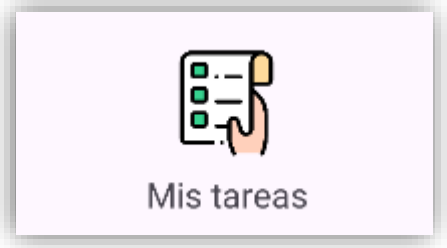


Fig. 5. Icono y logo de la app

```
ivLogo.setAlpha(0f); tvTitle.setAlpha(0f);
ivLogo.setScaleX(0f); tvTitle.setScaleX(0f);
ivLogo.setScaleY(0f); tvTitle.setScaleY(0f);

ivLogo.animate()
        .alpha( value: 1f)
        .scaleX( value: 1f)
        .scaleY( value: 1f)
        .rotation( value: 360f)
        .setDuration(2400)
        .start();
tvTitle.animate()
        .alpha( value: 1f)
        .scaleX( value: 1f)
        .scaleY( value: 1f)
        .rotation( value: 360f)
        .setDuration(2400)
        .start();
```

Fig. 6. Código para dar movimiento al logo y title, al inicio de la app

La interfaz incluye un campo de texto para introducir tareas, un botón para añadirlas, un listado mediante RecyclerView, un botón para eliminar todas las tareas y un icono de salida.

El diseño se ha realizado con XML y ConstraintLayout, incorporando elementos visuales como bordes, ajustes tipográficos y una barra de desplazamiento vertical personalizada para facilitar la navegación por la lista.

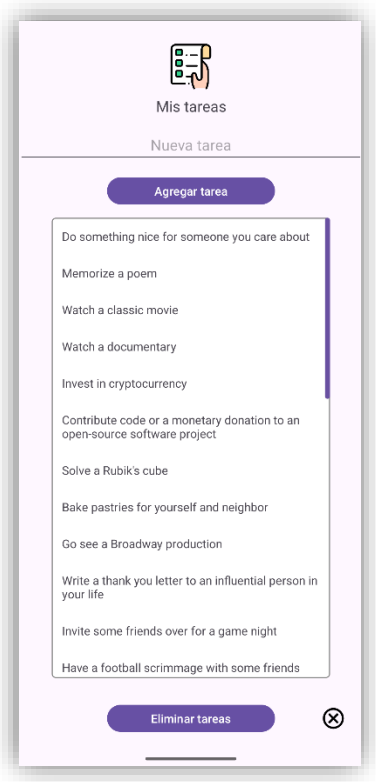


Fig. 7. Pantalla principal, al inicio carga la lista DummyJSON

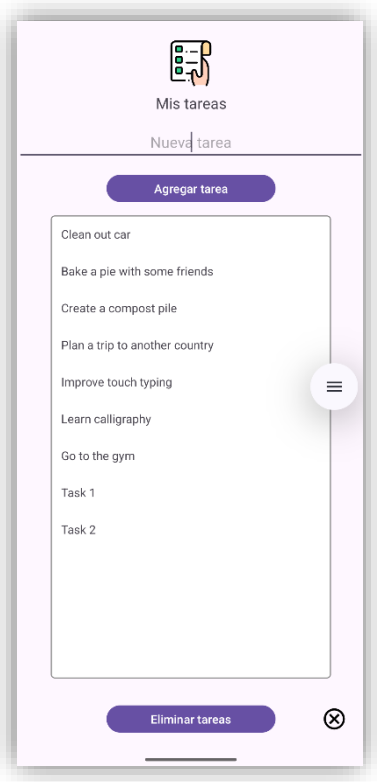


Fig.8. Lista modificada, eliminadas y creadas nuevas tareas

3.2. ADAPTACIÓN A LA ORIENTACIÓN DE PANTALLA

Se han desarrollado diseños diferenciados para las orientaciones vertical y horizontal mediante recursos XML específicos.

En orientación vertical, los componentes se distribuyen de forma secuencial para facilitar el uso en un teléfono móvil. En orientación horizontal, se ajustan los espacios, las dimensiones y los tamaños de texto para aprovechar la menor altura disponible y permitir visualizar más tareas simultáneamente.

La adaptación se realiza mediante el directorio layout-land, manteniendo la funcionalidad de los componentes en ambas orientaciones.



Fig.9. Pantalla horizontal

3.3. PERSONALIZACIÓN VISUAL

Se han incorporado recursos gráficos y ajustes de presentación para mejorar la experiencia de uso:

- Icono de la aplicación y título principal.
- Botones con dimensiones adaptadas al diseño.
- Bordes para delimitar visualmente la lista.
- Icono de salida en la parte inferior derecha.
- Ajustes del tamaño de texto para facilitar su visualización en horizontal.
- Personalización de la barra de desplazamiento vertical para hacerla más visible.

4. ARQUITECTURA Y FUNCIONALIDADES PRINCIPALES

4.1. ORGANIZACIÓN DE LAS TAREAS

Las tareas se gestionan mediante una colección de datos en Java y se muestran a través de un RecyclerView.

Para ello se utiliza un adaptador (TaskAdapter) que vincula los datos con las vistas de cada elemento. De esta forma, la lista puede actualizarse cuando se añaden o eliminan tareas.

4.2. AÑADIR TAREAS

El usuario introduce el texto en el EditText y pulsa el botón **Agregar tarea**.

La aplicación incorpora la nueva tarea a la lista y actualiza el RecyclerView para mostrar el cambio en la interfaz.

4.3. ELIMINAR UNA TAREA INDIVIDUAL

Se ha implementado un evento de pulsación larga (LongClick) sobre cada elemento de la lista.

Al mantener pulsada una tarea, esta se elimina individualmente y el listado se actualiza para reflejar la modificación.

4.4. ELIMINAR TODAS LAS TAREAS

El botón **Eliminar tareas** permite borrar el contenido completo de la lista.

Esta funcionalidad facilita la gestión del listado sin necesidad de eliminar cada elemento por separado.

4.5. NAVEGACIÓN A LA PANTALLA DE DETALLE

Al pulsar brevemente sobre una tarea, la aplicación abre una segunda pantalla mediante un Intent.

La tarea seleccionada se transmite a la actividad de detalle utilizando un dato adicional (Intent extra), lo que permite mostrar información correspondiente al elemento elegido.

Esta funcionalidad permite practicar la navegación entre actividades y el intercambio de datos dentro de una aplicación Android.

4.6. SALIDA DE LA APLICACIÓN

Se ha incorporado un icono de salida en la zona inferior derecha de la pantalla principal.

Este elemento permite implementar una acción de cierre mediante código Java, separando la presentación visual del comportamiento asociado al evento de pulsación.

5. CONSUMO DE LA API Y PERSISTENCIA DE DATOS

5.1. OBTENCIÓN DE TAREAS MEDIANTE DUMMYJSON

La aplicación utiliza una API REST para obtener tareas iniciales de ejemplo.

El proceso comprende las siguientes operaciones:

1. Realizar una petición HTTP al servicio web mediante Retrofit.
2. Recibir la respuesta del servidor en formato JSON.
3. Procesar los datos recibidos.
4. Incorporar las tareas obtenidas al listado que se muestra en la interfaz.

Este proceso permite aplicar los conceptos de comunicación entre una aplicación móvil y un servicio externo.

5.2. IMPLEMENTACIÓN DE LA PERSISTENCIA LOCAL

Para conservar los datos entre ejecuciones se utiliza SharedPreferences, junto con Gson para convertir la colección de tareas a formato JSON y recuperarla posteriormente.

El funcionamiento se basa en los siguientes pasos:

1. Convertir la lista de tareas a una cadena JSON.
2. Guardar dicha cadena en las preferencias locales mediante una clave identificativa.
3. Recuperar la información almacenada cuando se inicia la aplicación.
4. Convertir el JSON recuperado en una colección Java.
5. Actualizar el RecyclerView con los datos recuperados.

La persistencia debe actualizarse después de las operaciones que modifican la lista, como añadir o eliminar tareas. Asimismo, se contempla el comportamiento de la aplicación cuando la lista queda vacía, gestionando la carga automática de las tareas iniciales desde DummyJSON según la lógica implementada.

6. PRUEBAS Y VALIDACIÓN DEL PROYECTO

Durante la fase de pruebas se han comprobado las principales funcionalidades implementadas para verificar que la aplicación responde correctamente a las acciones del usuario. Las pruebas se han realizado tanto en el entorno de desarrollo como en un dispositivo móvil físico.

6.1. PRUEBAS FUNCIONALES

Las pruebas realizadas han permitido verificar el correcto funcionamiento de las siguientes funcionalidades:

- **Inicio de la aplicación:** la interfaz se muestra correctamente y las animaciones del título y del logotipo se ejecutan según lo previsto.
- **Gestión de tareas:** es posible añadir tareas, eliminar una tarea mediante una pulsación prolongada y eliminar todas las tareas mediante el botón correspondiente.
- **Persistencia local:** las tareas guardadas se conservan al cerrar y volver a abrir la aplicación.
- **Conexión con la API REST:** la aplicación obtiene correctamente los datos de DummyJSON.
- **Instalación y ejecución:** la versión de depuración (debug) se ha instalado y ejecutado satisfactoriamente en un dispositivo móvil físico.
- **Versión de lanzamiento:** la versión release también se ejecuta correctamente después de modificar la configuración de compilación.

6.2. CONFIGURACIÓN Y VALIDACIÓN DE LA VERSIÓN RELEASE

Durante las pruebas iniciales de la versión release se detectó un problema de funcionamiento relacionado con la optimización del código durante la compilación.

Para solucionarlo, se desactivó la optimización mediante la propiedad enable = false en la configuración de release. Esta modificación evita los problemas detectados durante las pruebas con Retrofit, la biblioteca utilizada para realizar las peticiones a la API REST.

Tras aplicar el cambio, se comprobó que la versión de lanzamiento funcionaba correctamente.

Aunque las pruebas realizadas han sido satisfactorias, antes de una eventual publicación en Google Play Store sería conveniente completar las comprobaciones finales y revisar los requisitos de distribución de la plataforma.

6.3. GENERACIÓN DE LA APK

Se ha generado correctamente el archivo de instalación de la aplicación en formato .apk mediante Android Studio, utilizando la variante de compilación **debug**.

La compilación se ha completado satisfactoriamente, obteniendo un paquete instalable que permite ejecutar y probar la aplicación en un dispositivo Android físico.

Además, se ha generado una versión **release** firmada, cuya ejecución se ha comprobado satisfactoriamente tras modificar la configuración de optimización de la compilación.

6.4. INSTALACIÓN EN UN DISPOSITIVO FÍSICO

La última versión debug de la aplicación se ha instalado y ejecutado satisfactoriamente en un dispositivo móvil físico.

Las pruebas han permitido comprobar el inicio de la aplicación, el funcionamiento de las principales interacciones, la navegación a la pantalla de detalle, las animaciones del título y del icono, la persistencia local y el comportamiento de la interfaz en las distintas orientaciones.

Los resultados obtenidos han confirmado el funcionamiento correcto de la versión final instalada.

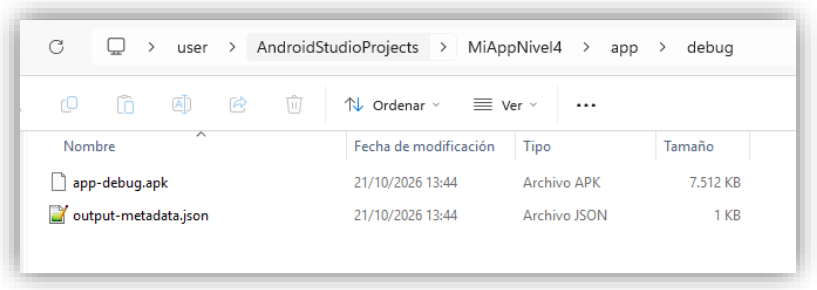


Fig.10. Archivo APK generado en la variante debug.

7. PREPARACIÓN DE LA APLICACIÓN PARA SU DISTRIBUCIÓN

En este apartado se describe el procedimiento necesario para preparar una versión distribuible de **MiAppNivel4** mediante Android Studio. Aunque la aplicación no se va a publicar, se documentan los pasos para generar un APK firmado y un Android App Bundle (AAB), formatos utilizados en la distribución de aplicaciones Android.

La versión debug generada durante las pruebas del proyecto permite instalar y ejecutar la aplicación en un dispositivo físico. Para preparar una versión destinada a su distribución, se utilizaría la variante **release** y se firmaría el paquete correspondiente

7.1. GENERACIÓN DE UN APK FIRMADO

El archivo APK (Android Package) contiene los elementos necesarios para instalar una aplicación Android. Una APK firmada puede distribuirse e instalarse en dispositivos compatibles, siempre que se cumplan las condiciones de instalación correspondientes.

Para generar una APK firmada desde Android Studio, se seguirían estos pasos:

1. Abrir el proyecto y acceder al menú Build y seleccionar la opción de generación de un paquete firmado, cuyo nombre puede variar según la versión de Android Studio.
2. En el asistente, seleccionar APK y continuar.
3. Seleccionar un almacén de claves (keystore) existente o crear uno nuevo.
4. Introducir la información solicitada para la firma y seleccionar la variante release.
5. Finalizar el asistente y esperar a que termine el proceso de compilación.
6. Localizar el archivo generado en la ruta indicada por Android Studio.
7. Si se quisiera distribuir la aplicación, instalar el paquete en un dispositivo de prueba y comprobar su funcionamiento.

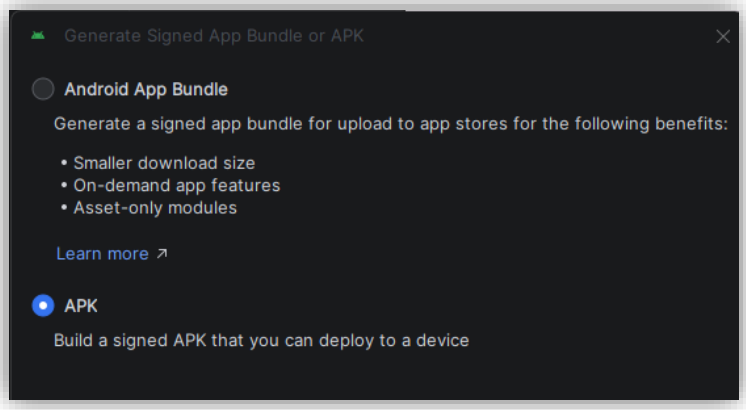


Fig.11. Asistente para generar APK firmada

El almacén de claves y sus contraseñas deben conservarse de forma segura, ya que son importantes para gestionar futuras actualizaciones firmadas de la aplicación. No deben compartirse ni incluirse en el repositorio de código.

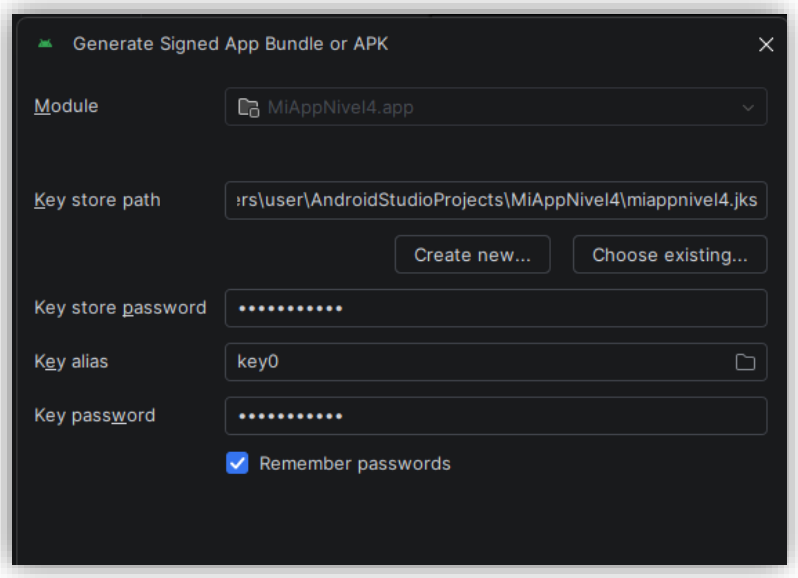


Fig.12. Selección de clave existente

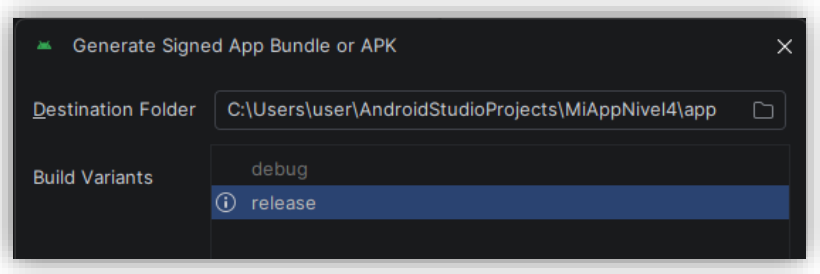


Fig.13. Selección variante release

7.2. GENERACIÓN DE UN ANDROID APP BUNDLE (AAB)

El formato Android App Bundle (.aab) es el formato habitual de entrega de aplicaciones nuevas en Google Play. A diferencia de un APK, no está diseñado para instalarse directamente en el dispositivo. Google Play utiliza el AAB para generar los APK optimizados que se distribuyen a los dispositivos compatibles.

Para generar un AAB firmado desde Android Studio, se seguirían estos pasos:

1. Abrir el proyecto MiAppNivel4 en Android Studio.
2. Acceder a Build → Generate Signed Bundle / APK.
3. Seleccionar Android App Bundle y continuar.
4. Seleccionar el almacén de claves existente o crear uno nuevo.
5. Introducir los datos de firma solicitados y seleccionar la variante release.
6. Elegir la ubicación de salida y finalizar el asistente.
7. Localizar el archivo .aab generado y conservarlo en una ubicación segura.

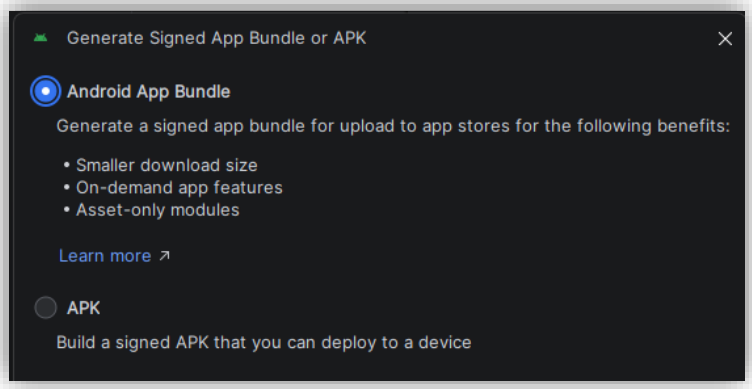


Fig.13. Asistente para generar APP Bundle firmada. El resto de los pasos del asistente son iguales que en caso anterior

Para una publicación real, el paquete debe estar firmado con la clave de carga y se debe configurar la firma de aplicaciones de Google Play.

Aunque en este proyecto no es necesario generar el AAB, puesto que no se va a publicar la aplicación, lo he generado como práctica del procedimiento.

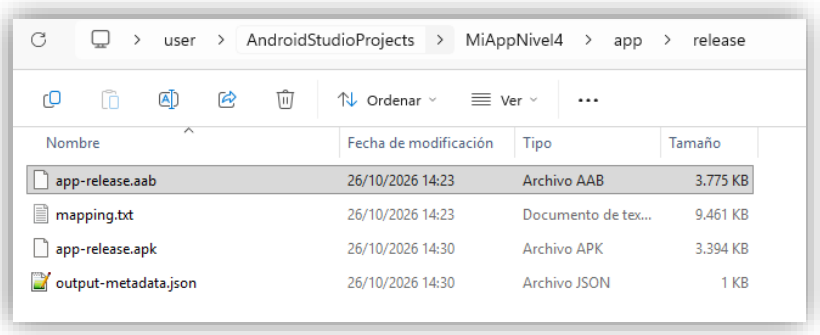


Fig.14. Archivos AAB y APK generados en la variante release

7.3. CONSIDERACIONES DE SEGURIDAD

Durante la preparación de una versión distribuible se deben tener en cuenta las siguientes medidas:

- Proteger el almacén de claves y las contraseñas utilizadas para firmar la aplicación.
- No incluir archivos de claves privadas ni contraseñas en el repositorio de GitHub.
- Conservar una copia de seguridad segura de las claves necesarias para futuras actualizaciones.
- Comprobar que la variante release se compila correctamente y probarla antes de distribuirla.

Estas medidas permiten proteger la aplicación y mantener la posibilidad de publicar actualizaciones firmadas en el futuro.

8. PROCEDIMIENTO DE PUBLICACIÓN EN GOOGLE PLAY CONSOLE

Google Play Console es la plataforma de Google que permite gestionar las aplicaciones Android destinadas a su distribución en Google Play. Desde ella se configura la ficha de tienda, se cargan los paquetes de distribución, se realizan pruebas y se gestiona el proceso de publicación.

Aunque MiAppNivel4 no se va a publicar, a continuación, se describe el procedimiento que se seguiría para preparar una futura distribución.

8.1. REGISTRO Y ACCESO A GOOGLE PLAY CONSOLE

El primer paso sería disponer de una cuenta de desarrollador de Google Play Console. Para ello, habría que completar el registro, los pagos que correspondan y las verificaciones solicitadas por Google.

Una vez registrado, se accedería a la plataforma mediante la cuenta autorizada.

8.2. CREACIÓN DE LA APLICACIÓN

Desde el panel principal se seleccionaría la opción Crear aplicación. A continuación, se indicarían los datos iniciales solicitados, como:

- Nombre de la aplicación: Mis Tareas.
- Idioma predeterminado: español.
- Tipo de aplicación: aplicación.
- Modalidad de distribución: gratuita o de pago, según la decisión que se adoptase.
- Dirección de correo electrónico de contacto.
- Aceptación de las declaraciones y condiciones correspondientes.

Después de completar los datos obligatorios, se crearía la ficha de la aplicación dentro de Play Console.

8.3. CUMPLIMENTACIÓN DE LOS REQUISITOS DE LA APLICACIÓN

Una vez creada, se revisarían los apartados que aparecen en el panel de configuración de Play Console.

Entre ellos pueden encontrarse:

- Información sobre el contenido de la aplicación.
- Clasificación por edades y contenido.
- Público objetivo.
- Declaración sobre la presencia de anuncios.
- Seguridad de los datos.
- Política de privacidad.
- Acceso a la aplicación, cuando sea necesario.
- Otros requisitos que Google determine según las características de la aplicación.

Las declaraciones deben cumplimentarse de forma veraz y de acuerdo con el comportamiento real de la aplicación y los servicios que utiliza. No sería correcto indicar que la aplicación no recopila datos sin comprobar antes su funcionamiento y las prácticas de los servicios integrados.

8.4. CARGA DEL PAQUETE DE DISTRIBUCIÓN

Cuando se hubieran completado los requisitos previos, se accedería a la sección de pruebas o publicación correspondiente y se crearía una nueva versión.

Para una aplicación nueva, el procedimiento habitual sería cargar el archivo AAB firmado generado desde Android Studio. Play Console validaría el paquete y mostraría los posibles errores o advertencias que debieran revisarse.

Antes de continuar, se comprobaría que el paquete corresponde a la versión deseada y que la información de la aplicación es correcta.

8.5. PRUEBAS PREVIAS A LA PUBLICACIÓN

Antes de distribuir la aplicación públicamente, se podría utilizar una pista de pruebas de Google Play, por ejemplo, de pruebas internas o cerradas.

Estas pruebas permitirían comprobar la instalación mediante Google Play, el funcionamiento de la aplicación en los dispositivos de prueba y la detección de posibles errores.

Los requisitos para acceder a producción dependen, entre otros factores, del tipo de cuenta de desarrollador. Por ejemplo, determinadas cuentas personales creadas después del 13 de noviembre de 2023 deben cumplir requisitos específicos de pruebas antes de publicar.

8.6. REVISIÓN Y PUBLICACIÓN

Finalmente, se revisarían todos los apartados obligatorios y los mensajes de validación de Play Console. Cuando se cumplieran los requisitos aplicables, se podría enviar la versión a revisión.

La distribución pública quedaría condicionada a la aprobación de Google y a la decisión expresa de publicar la aplicación.

En el caso de MiAppNivel4, no se realizará la carga del paquete, el envío a revisión ni la publicación en Google Play. Este apartado tiene una finalidad exclusivamente documental y formativa.

9. PREPARACIÓN DE LA FICHA DE GOOGLE PLAY

La ficha de tienda es la información que los usuarios consultan para conocer una aplicación antes de instalarla. Su preparación incluiría el nombre, las descripciones, el icono, las capturas de pantalla y los demás recursos solicitados por Google Play.

Aunque MiAppNivel4 no se va a publicar, se propone la siguiente ficha como ejercicio de documentación, ajustada a las funciones desarrolladas.

9.1. DATOS GENERALES

Elemento	Propuesta
Nombre de la aplicación	Mis Tareas
Idioma	Español
Tipo	Aplicación Android
Categoría	Productividad, si resulta adecuada a la clasificación disponible
Modalidad	Gratuita, como propuesta para una hipotética distribución
Público objetivo	Se definiría según el público al que se destinara la aplicación

Estos datos son una propuesta orientativa, no una ficha creada ni registrada en Google Play Console.

9.2. DESCRIPCIONES DE LA APLICACIÓN

Descripción breve

Organiza tus tareas personales, añade nuevas, consulta sus detalles y conserva tu lista en el dispositivo.

Descripción completa

MiAppNivel4 es una aplicación Android desarrollada en Java como ejercicio práctico del curso Desarrollo de Aplicaciones Móviles: Nivel 4.

La aplicación permite gestionar una lista de tareas personales mediante una interfaz sencilla e interactiva.

Entre sus funcionalidades se incluyen:

- Añadir nuevas tareas a la lista.
- Consultar el detalle de una tarea seleccionada.
- Eliminar tareas mediante las interacciones disponibles.
- Conservar las tareas mediante almacenamiento local.
- Consultar información mediante la API de DummyJSON.
- Utilizar una interfaz con animaciones y adaptación a distintas orientaciones del dispositivo.

El proyecto permite aplicar conocimientos de desarrollo Android, navegación entre actividades, persistencia local, consumo de servicios web y diseño de interfaces.

9.3. RECURSOS GRÁFICOS

Para preparar una ficha de tienda, sería necesario reunir los recursos gráficos requeridos por Google Play y comprobar sus dimensiones y formatos vigentes antes de utilizarlos.

Se podrían preparar los siguientes elementos:

- Icono representativo de una aplicación de gestión de tareas, basado en una lista de tareas. Se utilizaría un diseño sencillo y reconocible, adaptado a las especificaciones de Google Play Store: imagen cuadrada de 512 × 512 píxeles, formato PNG de 32 bits y tamaño máximo de 1 MB.



Fig.15. Icono

- Pantalla principal en orientación vertical: mostraría el título, el campo de entrada, los controles y la lista de tareas.

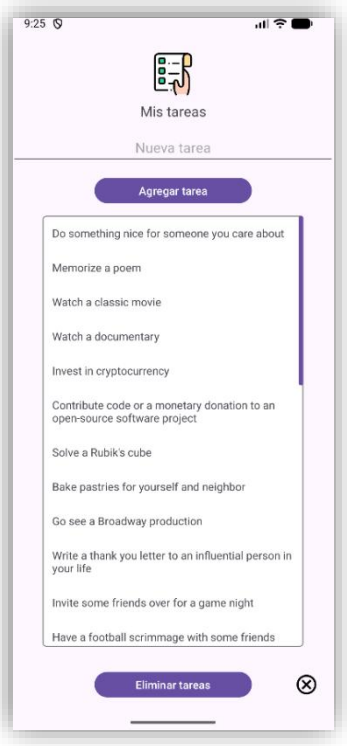


Fig.16. Pantalla principal vertical

- Pantalla principal en orientación horizontal: permitiría mostrar la adaptación de la interfaz al cambiar la orientación del dispositivo.

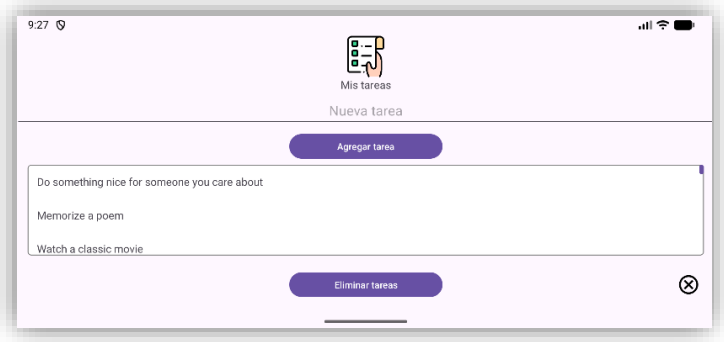


Fig.17. Pantalla principal horizontal

- Pantalla de detalle: mostraría la información correspondiente a una tarea seleccionada.

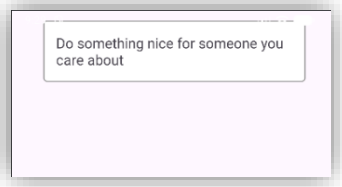


Fig.18. Pantalla de detalle

9.4. INFORMACIÓN LEGAL Y PRIVACIDAD

Antes de una publicación real, habría que revisar la política de privacidad, la información sobre seguridad de los datos y las demás declaraciones exigidas por Google Play.

La información debe corresponder al funcionamiento real de la aplicación, incluido el uso de almacenamiento local y los servicios externos que intervienen. Si la aplicación se distribuyera, también habría que comprobar qué datos se transmiten a DummyJSON y qué tratamiento de datos realizan los servicios integrados.

Como este proyecto no se va a publicar, no es necesario crear una ficha definitiva ni enviar declaraciones a Google Play Console.

10. CONCLUSIÓN

El desarrollo de **MiAppNivel4**, presentada al usuario como **Mis Tareas**, ha permitido aplicar de forma práctica los principales conceptos del desarrollo de aplicaciones móviles con Android Studio y Java.

Durante el proyecto se han implementado interfaces mediante XML y ConstraintLayout, listas dinámicas con RecyclerView, adaptadores, navegación entre actividades e intercambio de información mediante Intent.

La aplicación integra el consumo de una API REST mediante Retrofit para obtener tareas iniciales de DummyJSON y utiliza Gson junto con SharedPreferences para almacenar y recuperar los datos localmente. Las pruebas realizadas han confirmado el correcto funcionamiento de la persistencia, incluida la recuperación de las tareas al volver a iniciar la aplicación.

Asimismo, se han incorporado animaciones en el título y el logotipo, interacciones mediante clic y pulsación larga, eliminación individual y total de tareas, y diseños adaptados a las orientaciones vertical y horizontal del dispositivo.

Por último, se ha generado el archivo APK y se ha comprobado satisfactoriamente el funcionamiento de la última versión en un dispositivo físico.

En conclusión, el proyecto ha permitido integrar y poner en práctica conocimientos de interfaz gráfica, gestión de eventos, comunicación con servicios web, persistencia local y adaptación visual, obteniendo como resultado una aplicación Android funcional y comprobada.